

1.To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define MAX_HISTORY 10
#define MAX_BUFFER 100

char history[MAX_HISTORY][MAX_BUFFER];
int history_count = 0;

/* Enable terminal raw mode */
void enable_raw_mode(struct termios *original)
{
    struct termios raw;

    tcgetattr(STDIN_FILENO, original);

    raw = *original;

    raw.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

/* Restore normal terminal mode */
void disable_raw_mode(struct termios *original)
{
    tcsetattr(STDIN_FILENO, TCSAFLUSH, original);
}

/* Store command in history */
void store_history(char *command)
{
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 0; i < MAX_HISTORY - 1; i++)
        {
            strcpy(history[i], history[i + 1]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

/* Display history */
void display_history()
{
    printf("\n\nCommand History:\n");

    for (int i = 0; i < history_count; i++)
    {
        printf("%d: %s\n", i + 1, history[i]);
    }
}

/* Read command with arrow-key support */
void read_command(char *buffer)
{
    struct termios original;

    int position = 0;
    int history_position = history_count;

    char ch;

    enable_raw_mode(&original);

    while (1)
    {
        read(STDIN_FILENO, &ch, 1);

        /* Enter key */
        if (ch == '\n' || ch == '\r')
        {
            buffer[position] = '\0';
            printf("\n");
            break;
        }

        /* Backspace */
        else if (ch == 127)
        {
            if (position > 0)
            {
                position--;
                buffer[position] = '\0';

                printf("\b \b");
                fflush(stdout);
            }
        }
    }
}
```

```

/* Escape sequence */
else if (ch == 27)
{
    char seq[2];

    read(STDIN_FILENO, &seq[0], 1);
    read(STDIN_FILENO, &seq[1], 1);

    /* Up Arrow */
    if (seq[0] == '[' && seq[1] == 'A')
    {
        if (history_position > 0)
        {
            history_position--;

            /* Clear current input */
            while (position > 0)
            {
                printf("\b\b");
                position--;
            }

            strcpy(buffer, history[history_position]);
            position = strlen(buffer);

            printf("%s", buffer);
            fflush(stdout);
        }
    }

    /* Down Arrow */
    else if (seq[0] == '[' && seq[1] == 'B')
    {
        if (history_position < history_count - 1)
        {
            history_position++;

            while (position > 0)
            {
                printf("\b\b");
                position--;
            }

            strcpy(buffer, history[history_position]);
            position = strlen(buffer);

            printf("%s", buffer);
            fflush(stdout);
        }
    }
}

```

```

    else
    {
        history_position = history_count;

        while (position > 0)
        {
            printf("\b\b");
            position--;
        }

        buffer[0] = '\0';
    }
}

/* Normal character */
else if (ch >= 32 && ch <= 126)
{
    if (position < MAX_BUFFER - 1)
    {
        buffer[position++] = ch;
        buffer[position] = '\0';

        printf("%c", ch);
        fflush(stdout);
    }
}

}

disable_raw_mode(&original);
}

int main()
{
    char buffer[MAX_BUFFER];

    printf("=== Command History Program ===\n");
    printf("Type commands and press Enter.\n");
    printf("Use UP/DOWN arrows for history.\n");
    printf("Type 'history' to display history.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        read_command(buffer);

        if (strcmp(buffer, "exit") == 0)
        {
            break;
        }
    }
}

```

```

}

if (strcmp(buffer, "history") == 0)
{
    display_history();
    continue;
}

store_history(buffer);

printf("Command entered: %s\n", buffer);
}

printf("\nProgram terminated.\n");

return 0;
}

```

```

qowtham@DESKTOP-2F03K3T:~/OS$ nano command_history.c
qowtham@DESKTOP-2F03K3T:~/OS$ gcc command_history.c -o command_history
qowtham@DESKTOP-2F03K3T:~/OS$ ./command_history
=== Command History Program ===
Type commands and press Enter.
Use UP/DOWN arrows for history.
Type 'history' to display history.
Type 'exit' to quit.

myshell> ls
Command entered: ls
myshell> pwd
Command entered: pwd
myshell> date
Command entered: date
myshell> whoami
Command entered: whoami
myshell> myshell> whoami
Command entered: myshell> whoami
myshell> whoami
Command entered: whoami
myshell> date
Command entered: date
myshell> pwd
Command entered: pwd
myshell> ls
Command entered: ls
myshell> history

Command History:
1: ls
2: pwd
3: date
4: whoami
5: myshell> whoami
6: whoami
7: date
8: pwd
9: ls
myshell> ls|

```

PART 2 — Dynamic Memory Management

```

qowtham@DESKTOP-2F03K3T:~/OS$ nano dynamic_memory.c
qowtham@DESKTOP-2F03K3T:~/OS$ gcc dynamic_memory.c -o dynamic_memory
qowtham@DESKTOP-2F03K3T:~/OS$ ./dynamic_memory
Dynamic Array Demonstration
Array resized to 10 elements

Array contents:
10 20 30 40 50 60 70 80 90 100

Linked List: 50 -> 40 -> 30 -> 20 -> 10 -> NULL

All dynamically allocated memory released.
qowtham@DESKTOP-2F03K3T:~/OS$ |

```

PART 3 — Verify Using Valgrind

```
gowtham@DESKTOP-2F03K3T:~/OS$ valgrind --version
valgrind-3.26.0
gowtham@DESKTOP-2F03K3T:~/OS$ gcc dynamic_memory.c -o dynamic_memory
gowtham@DESKTOP-2F03K3T:~/OS$ gcc dynamic_memory.c -o dynamic_memory
gowtham@DESKTOP-2F03K3T:~/OS$ gcc -g dynamic_memory.c -o dynamic_memory
gowtham@DESKTOP-2F03K3T:~/OS$ valgrind --leak-check=full ./dynamic_memory
==1817== Memcheck, a memory error detector
==1817== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1817== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1817== Command: ./dynamic_memory
==1817==
Dynamic Array Demonstration
Array resized to 10 elements

Array contents:
10 20 30 40 50 60 70 80 90 100

Linked List: 50 -> 40 -> 30 -> 20 -> 10 -> NULL

All dynamically allocated memory released.
==1817==
==1817== HEAP SUMMARY:
==1817==    in use at exit: 0 bytes in 0 blocks
==1817== total heap usage: 8 allocs, 8 frees, 1,164 bytes allocated
==1817==
==1817== All heap blocks were freed -- no leaks are possible
==1817==
==1817== For lists of detected and suppressed errors, rerun with: -s
==1817== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
gowtham@DESKTOP-2F03K3T:~/OS$ |
```